AForce OS — Expanded Claim Hooks

AForce OS — Expanded Claim Hooks

Companion to: `aforce-os-engineering-brief.md` **Audience:** Outside counsel preparing the provisional specification **Date:** April 29, 2026

This document expands the five candidate claim hooks listed in §7 of the engineering brief. For each hook it provides:

1. A one-line **summary** (what the claim covers).
2. A **field of art** sentence usable in the background.
3. The **technical elements** (limitations) as engineers see them, written in numbered form so counsel can convert to claim language.
4. **Implementation evidence** — the exact files and line ranges that support the limitations.
5. A short **why-novel** note explaining how this differs from existing apps.
6. Suggested **dependent-claim seeds**.

This is engineering input only and is **not** legal advice. The claim language, scope, transitional phrases ("comprising" vs. "consisting of"), and the question of whether each element is in fact patentable, are decisions for counsel.

Hook 1 — Pure-function performance scoring with audit-on-output

Summary. A scoring system that computes a numeric performance score for a hydration / recovery user from a defined snapshot of inputs, where the same input snapshot always produces the same output, and where every score returned to the UI is accompanied by a structured breakdown of bounded contributions.

Field of art sentence. Mobile health and performance applications increasingly compute a single composite score (hydration, readiness, recovery) from a mix of biometric and behavioral inputs. Existing systems treat the score as opaque to the user and frequently rely on probabilistic / ML inference, which is not reproducible and not auditable.

Technical elements.

1. A user-state snapshot data structure containing at least: cumulative fluid intake on a per-event basis with timestamps; cumulative AForce-formulated unit count; environmental load (heat, humidity, activity); a physiological self-report axis; one or more biometric samples (HRV, sleep) read from a device-level health framework; and a boolean operating-mode flag (e.g. clutch / social).
2. A pure-function score engine that consumes the snapshot and produces a numeric score in a fixed range (e.g. 0 – 100) by summing a defined set of bounded contribution terms.
3. A continuous-time decay model applied to the score between intake events, parameterized by user mass, activity, and environment.
4. A breakdown array, returned alongside the score on every invocation, in which each entry comprises an identifier, a human-readable label, a signed delta, and a maximum-magnitude bound, such that no single contribution can exceed its bound.
5. Server-side and client-side execution of the same engine on the same snapshot, producing the same score and the same breakdown (a "score identity invariant").

Implementation evidence.

- Score function: `artifacts/aforce-os/utils/scoringEngine.ts:363` (sum-of-terms), `:367` (clamp 0–100).
- Decay function: `artifacts/aforce-os/utils/scoringEngine.ts:166–202`.
- Breakdown shape: `artifacts/aforce-os/types/index.ts` (`ScoreContribution`).
- Per-event absorption (basis for bounded base term): `artifacts/aforce-os/services/hydrationScoreService.ts:25–86`.
- Server-replay invariant: `artifacts/aforce-os/services/realApi.ts` re-runs the same `scoringEngine` on every API response.

Why-novel. Conventional scoring apps return a number with no explanation, or rely on LLM / ML inference whose output cannot be re-derived from the inputs after the fact. The audit-on-output pattern means every score the user sees can be reproduced offline by replaying the snapshot, which is a property required for safety-class behavior (see Hook 5) and for a meaningful "Why?" UI.

Dependent-claim seeds.

- The system of Hook 1 wherein the breakdown is rendered to the user as a per-term magnitude bar chart.
- The system of Hook 1 wherein each contribution term is derived from a single named input axis, such that the user can trace any score change to a specific change in their behavior.
- The system of Hook 1 wherein the decay model is multiplied by an operating-mode multiplier when the boolean operating-mode flag is set.

Hook 2 — Sweat-deficit-driven cadence override (Autopilot)

Summary. A method for replacing a band-based recheck timer with a physiologically-derived recheck interval for a bounded recovery window after a measured sweat session.

Field of art sentence. Hydration coaching apps prompt the user to log fluid intake at fixed or rule-of-thumb intervals (e.g. every 30 minutes). These intervals are not aware of the user's actual fluid loss during a measured activity, and therefore systematically under-prompt high-deficit athletes during the post-activity period when corrective intake matters most.

Technical elements.

1. A sweat-rate engine that consumes a session description (mode, intake, output, environment, duration) and produces at least: a sweat rate, a hydration deficit expressed as a percentage of body mass, and a sodium-loss estimate.
2. A cadence-derivation function that maps the deficit percentage to an `(interval, urgency, window)` tuple via a stepped table: a first deficit threshold mapping to a short interval and a critical urgency; a second, lower threshold mapping to a longer interval and a high urgency; otherwise the longest interval and a moderate urgency. The window is fixed.
3. An atomic state-transition action ("set autopilot") that, on dispatch, simultaneously: stores the autopilot tuple and a timestamp; resets a visible recheck-countdown to `interval × 60` seconds; and clears any pending user-confirmation prompt.
4. A consumer hook that, on each evaluation tick, checks whether the autopilot timestamp lies within the recovery window; if so, returns the autopilot interval and urgency in place of the band-default cadence; if not, returns the band-default cadence.
5. Display of the resulting countdown in the same UI element regardless of source, such that the user experience does not change when the cadence source switches.

Implementation evidence.

- Sweat engine + cadence derivation: `artifacts/aforce-os/services/sweatRateEngine.ts:489–493` (`deriveAutopilot`).
- Atomic state transition: `artifacts/aforce-os/store/appStoreReducer.ts:15–35` (`SET_SWEAT_AUTOPILOT`).
- Consumer hook: `artifacts/aforce-os/hooks/useHeatGuard.ts:106–117` .
- UI surface: `artifacts/aforce-os/components/RiskTimerDisplay.tsx` .

Why-novel. No consumer hydration app the team is aware of overrides its recheck cadence with a per-session-derived interval ladder. Existing products use either fixed timers or simple rule-of-thumb heuristics that do not consume a measured deficit %. The atomic reset of the *visible* timer at the moment of dispatch — so the user immediately sees the new cadence — is also non-trivial: it requires the cadence source and the timer source to be co-mutated in a single transition.

Dependent-claim seeds.

- The method of Hook 2 wherein the cadence source falls back to the band-default cadence after the recovery window expires, without an explicit exit step.
- The method of Hook 2 wherein the recovery window is four hours.
- The method of Hook 2 wherein the cadence-derivation function uses thresholds of approximately 4 %

and 2 % deficit, mapping to intervals of 8, 12, and 20 minutes respectively.

Hook 3 — Live-state beverage-replacement recommendation (Autoscan)

Summary. A barcode-driven beverage-recognition system that scores a scanned product not against a static nutrition database but against a contemporaneously computed user state, and that surfaces a brand-specific replacement recommendation when an in-catalog brand item exceeds the scanned product's fit score by a configurable margin.

Field of art sentence. Existing barcode-scanning beverage apps return static nutrition facts (calories, sugar, sodium) regardless of the user's current physiological state. They do not score a scanned product against a live computation of what the user actually needs in the moment, and they do not recommend a specific replacement product.

Technical elements.

1. A camera-and-barcode subsystem capable of resolving common UPC / EAN symbologies and a brand-specific QR scheme; the resolution layer is augmented by an external public-database fallback when a barcode is not present in the local catalog.
2. A normalized product schema with at least: product identity, brand, category, hydration speed, electrolyte density, sugar content, and an `isBrand` flag identifying products from the application's own brand catalog.
3. A comparison engine that takes the resolved product, a snapshot of the user's current state (the same snapshot consumed by the scoring engine of Hook 1), and produces a fit score (0 – 100) and a discrete verdict.
4. A replacement-decision rule: filter the product catalog to brand items only; compute the best brand item's fit score against the same user state; if `(brandFitScore − scannedFitScore) > T` (a configurable margin), return a replacement card naming the brand item and a one-line user-actionable command.
5. A result UI that displays both the scanned product and (conditionally) the replacement card, with a single tap to log either choice as an `IntakeEvent`.

Implementation evidence.

- Recognition + OFF fallback: `artifacts/aforce-os/services/productRecognitionService.ts:126`.
- Scan orchestrator: `artifacts/aforce-os/services/hydrationScanService.ts:55–101` (efficiency, fit, replacement).
- Comparison engine: `artifacts/aforce-os/services/comparisonEngine.ts`.
- Camera UI: `artifacts/aforce-os/components/CameraScanModal.tsx:38–48`.

Why-novel. A live-state replacement is a different user experience from a static-facts lookup: the recommendation depends on the user's current heat load, last intake, urine signal, etc., not just on the scanned product. It requires the comparison engine and the live state to share a snapshot type, which in turn requires the scoring system of Hook 1.

Dependent-claim seeds.

- The method of Hook 3 wherein the configurable margin `T` is approximately 4 fit-score points.
- The method of Hook 3 wherein the brand-specific QR scheme uses the URI form `<scheme>://product/<id>`.
- The method of Hook 3 wherein the result UI presents the replacement card only when the scanned product is not itself an in-brand item.

Hook 4 — Mode-priority decision pipeline with safety-class preemption

Summary. A decision pipeline that classifies a user into one of a defined set of operating modes, where a "safety-class" mode (e.g. alcohol-impairment) preempts the standard performance pipeline and emits

commands the rest of the engine cannot override.

Field of art sentence. Wellness apps that issue user-facing guidance commonly use a single decision tree across all states. A user who is intoxicated still receives the same hydration nudge as a sober athlete. There is no architectural mechanism for a safety-related command to take precedence over a routine command.

Technical elements.

1. A classification step that, given the user-state snapshot, identifies whether a safety-class mode is active (in the implementation: `socialMode.active === true`).
2. A safety-class command generator that, when the safety-class mode is active, runs a parallel ruleset whose outputs cannot be overridden by the standard ladder. In the implementation: BAC estimation via the Widmark approximation, impairment-level mapping to one of five tiers, and an escalation table that selects from a set of locked, globally-safe command strings.
3. A standard-mode command generator that runs only when the safety-class mode is *not* active, selecting commands from a band-keyed table.
4. A pipeline order in which the safety-class generator runs first; if it returns a non-null command, that command is the engine's output, and the standard generator is not consulted.
5. A locked-copy registry for safety-class command strings, separated from the localized template store, such that the disclaimer copy cannot be edited without an explicit code-level change.

Implementation evidence.

- Pipeline order: `artifacts/aforce-os/utils/scoringEngine.ts:537–544` (Social Mode runs first; non-null return short-circuits the band ladder).
- Safety-class generator: `artifacts/aforce-os/utils/scoringEngine.ts:458–531`.
- BAC + impairment: `artifacts/aforce-os/services/bacEstimationService.ts:92–147`, `artifacts/aforce-os/services/legalSafetyService.ts:28–37`.
- Standard generator: `artifacts/aforce-os/utils/scoringEngine.ts:557–590`.

Why-novel. The architectural commitment that a safety-class command cannot be overridden by a routine command is what makes this differ from a generic if/else inside a recommendation function. The locked-copy registry and the deterministic pipeline order are concrete mechanisms enforcing the preemption.

Dependent-claim seeds.

- The method of Hook 4 wherein the safety-class mode is alcohol-impairment, classified by a Widmark-approximation BAC computation.
- The method of Hook 4 wherein the safety-class command set includes at least one transportation-safety command and at least one stop-intake command.
- The method of Hook 4 wherein the standard generator's recheck cadence is itself overridable by a third source (the Autopilot of Hook 2), but the safety-class command is not.

Hook 5 — Sweat-derived sodium audit and intentional sodium-gap framing

Summary. A method for computing, at the conclusion of a measured sweat session, both an estimate of sodium loss and an estimate of sodium delivered by the recommended brand intake, and for surfacing the difference as an *intentional gap* attributed to the brand's formulation strategy rather than as a deficit to be closed by additional sodium.

Field of art sentence. Existing electrolyte-replacement guidance recommends matching sodium loss with sodium intake in a 1 : 1 ratio. Marketed electrolyte products compete on sodium quantity (e.g. 1000 mg / serving). This positions sodium quantity as the primary axis of recovery quality, which is not consistent with cellular-uptake research the brand cites.

Technical elements.

1. A sweat-rate engine that produces, in addition to a fluid-loss estimate, a sodium-loss estimate `sodiumLossMg`.
2. A brand-product sodium constant, `BRAND_SODIUM_PER_UNIT_MG`, the value of which is intentionally

small relative to comparable marketed electrolyte products.
3. A prescription generator that sizes the number of brand servings recommended by *fluid budget*, not by sodium-matching, and that exposes the resulting `brandSodiumTotalMg = servings × BRAND_SODIUM_PER_UNIT_MG`.
4. A computation `sodiumGapMg = max(0, sodiumLossMg − brandSodiumTotalMg)`, surfaced to the user with framing that attributes the gap to non-sodium components of the brand formulation (the implementation uses "structured-water absorption + marine minerals").
5. A comparison-table UI that displays the brand sodium-per-serving alongside two or more comparable products' sodium-per-serving, anchored by a closer line stating that "more sodium is not always the goal."

Implementation evidence.

- Sodium constant: `artifacts/aforce-os/services/sweatRateEngine.ts:128`.
- Audit math: `artifacts/aforce-os/services/sweatRateEngine.ts:506–511`.
- UI surface (Recovery Intelligence + Comparison Table): `artifacts/aforce-os/screens/SweatCalculatorScreen.tsx` (cards B and H of the 9-card stack).

Why-novel. The novelty is the inversion of the standard sodium-matching paradigm into an *intentional gap* that is calculated, surfaced, and explained as a product-design choice. The implementation produces a numeric value (`sodiumGapMg`) and renders it in a way that the user can audit; this is different from marketing copy that simply asserts "less sodium is enough."

Dependent-claim seeds.

- The method of Hook 5 wherein `BRAND_SODIUM_PER_UNIT_MG` is approximately 25 mg.
- The method of Hook 5 wherein the prescription generator sizes brand servings by a per-serving fluid budget of approximately 12 oz.
- The method of Hook 5 wherein the comparison-table UI is a card in a multi-card result pane that also includes the sodium-gap audit and a recovery-protocol recommendation gated by user-held inventory of the brand product.

Cross-hook dependencies

Several hooks reinforce one another and may strengthen the application if drafted together:

- Hook 2 (Autopilot) consumes the live state defined in Hook 1 and is consumed by the timer surface that Hook 4's mode-priority pipeline writes to.
- Hook 3 (Autoscan) consumes the same `UserState` snapshot type as Hook 1 — the live-state property is what makes Autoscan's recommendation different from a static lookup.
- Hook 5 (sodium gap) is the user-visible payoff of the sweat session that Hook 2 uses to derive Autopilot.

Counsel may consider drafting one independent claim per hook and then citing the cross-hook dependencies as supporting language in dependent claims.

Items the team did not include and why

The team is intentionally not surfacing the following as candidate claims, but counsel may consider them on review:

- **The choice of specific weights inside the score formula** (e.g. `0.4 × weight/150`). The numeric weights are tunable parameters; the team's view is that the *structure* of the formula (Hook 1) is more defensible than any one weight value. Counsel may decide otherwise.
- **The Widmark equation itself.** Widmark is prior art (1932). What is novel here is its embedding inside the safety-class preemption pipeline (Hook 4), not the equation.
- **The use of Apple HealthKit per se.** HealthKit is a public OS API. The novel element is the bounded-contribution way HealthKit signals enter the score formula (Hook 1), not the act of reading from HealthKit.

End of expanded hooks. Engineering is available to provide working-code walk-throughs of any hook on request.